

Dexterous Manipulation in MuJoCo: RUKA Hand on Piper Arm

PathOn Robotics Take-Home Project — Technical Report

Anish Yalavarthi · May 10, 2026

1. Overview

In this report I summarize the end-to-end dexterous-grasping pipeline I built in MuJoCo: the NYU RUKA 5-finger underactuated hand (15 DoF) mounted on the AgileX Piper 6-DoF collaborative arm, picking YCB objects from a tabletop using 3D-vision-based grasp prediction. I organized the work into five main pieces:

1. **Combined robot model** (`piper_ruka.xml`) — I re-parented the RUKA palm onto Piper’s `link6` body with a **10 cm** offset along +Z and a single 180° rotation around the diagonal axis documented in `docs/task1_mount.md`.
2. **Tabletop scene with cameras** — I authored `assets/scene/full_scene.xml` with a table, pedestal, five YCB objects, and three D435-like RGB-D cameras; I render depth and fuse point clouds with `scripts/render_scene.py`.
3. **3D vision-based grasp prediction** — I implemented a heuristic geometric pipeline (PCA OBBs, YAML grasp presets, ranked proposals) in `src/grasping/grasp_proposal.py`, with visualization in `scripts/visualize_grasps.py`.
4. **Planning and execution** — I run multi-waypoint IK on the palm site using damped least-squares (`src/planning/ik_solver.py`) and a state machine in `src/planning/executor.py`: `home -> pre-grasp -> intermediate -> grasp -> close -> lift`.
5. **Demo video** — I batch-record all objects with `scripts/record_all_grasps.py` (quad-view mosaic, H.264 MP4 via `imageio[ffmpeg]`). The submission clip is **on YouTube**; `docs/task5_demo.md` documents reproduction and how it aligns with the Task 4 evaluation.

The submitted bundle contains my code, diagnostic scripts, per-task notes under `docs/`, and the technical report (`report/`).

2. Approach

2.1 Combined robot model

I sourced the Piper MJCF from the MuJoCo Menagerie and the RUKA MJCF from the official RUKA repository. I edited both before I could combine them:

- **Piper:** I removed the default 2-finger gripper (link7/link8), the gripper actuator, and the inter-finger equality constraint.
- **RUKA:** I removed the Y-up gravity override (RUKA was authored under a different world convention), the dummy `world_frame` wrapper, and a degenerate slide joint.

The mount transform — the kinematically critical quantity that bridges my perception pipeline’s “hand frame” and my planning pipeline’s “flange frame” — is:

```
pos  = (0, 0, 0.10)           // 10 cm along link6 +Z
quat = (0, 0.707, 0.707, 0)   // 180° around (1,1,0)/√2 in link6 frame
```

This rotation maps RUKA’s $-Z$ (finger direction) onto link6 $+Z$ (tool axis), and orients the palm to face “down” toward the workspace at Piper’s home pose — a configuration I chose so that top-down grasps need less joint-6 rotation.

I verified the mount at three independent levels: symbolic (exact rotation matrix from the quaternion), numerical (forward kinematics at home), and physics (active contact pairs). All three pass; the mount is correct to within finger splay in the numerical view ($\sim 6.6^\circ$) and to machine precision in the symbolic view (0.000°).

2.2 Scene and sensors

In `assets/scene/full_scene.xml` I place the Piper+RUKA on a 70cm pedestal so its base is level with a 60×80 cm table at world $(0.5, 0, 0.7)$. I place five YCB objects (banana, mug, cracker box, mustard bottle, tennis ball) on the table with at least 18cm clearance between neighbors, chosen for grasp-type diversity (precision/wrap/box/lateral/spherical). I assign each object physically plausible mass, an inertia estimate scaled to its geometry, and convex-decomposed collision meshes (4-5 pieces per object) for stable contact resolution.

My camera configuration is **hybrid (two scene-mounted RGB-D + one wrist-mounted RGB-D)**. All use RealSense D435-like intrinsics — 640×480 resolution, 58° vertical FOV, depth in 0.1–3m. I place `scene_cam` at $(-0.3, 0.7, 1.7)$ (back-left) and `scene_cam2` at $(1.0, -0.5, 1.5)$ (front-right), both targeting the table center. I added the second scene camera after I saw partial clouds and biased OBBs with a single view; the fused cloud grew to ~ 620 k points and my OBBs centered better. **`scene_cam` and `scene_cam2` are the primary cameras** that drive Tasks 2–4 — together they provide $\sim 95\%$ surface coverage of the objects on the table. I also mount `wrist_cam` on the back of the RUKA palm looking along the finger direction, but in this submission it is wired up for parity with real eye-in-hand setups rather than actively used: at the home pose it stares at the sky ($\sim 13\%$ valid depth) and the heuristic perception path does not consume it. I left it defined so visual-servoing could be a drop-in addition later.

I wrote `scripts/render_scene.py` to iterate over all cameras and produce, per camera: RGB PNG, raw float32 depth, colorized depth visualization, and a world-frame colored point cloud (.ply). I also emit a fused multi-camera point cloud and a `cameras.json` with all intrinsic and extrinsic matrices. I apply the MuJoCo-to-OpenCV camera convention flip (`diag(1, -1, -1)` on the rotation matrix) at calibration export time so my Task 3 code and point-cloud libraries get standard-convention extrinsics.

2.3 Grasp model choice

I chose a **heuristic / analytical pipeline** over learned models (AnyDexGrasp, DexGraspNet, UniDexGrasp). Three factors drove my choice:

1. **Hand topology mismatch:** SOTA learned grasp models target Allegro / Shadow / MANO hands. Retargeting to RUKA’s 15-DOF tendon-driven topology is a nontrivial research problem that would have dominated my timeline.
2. **Incremental verifiability:** each pipeline stage (segmentation, OBB, palm pose, finger preset) produces a testable intermediate; learned models felt end-to-end or nothing for the time I had.
3. **Task fit:** my five YCB objects are well-characterized shapes (cylinder, box, sphere) for which geometric reasoning is both sufficient and well-studied (Cutkosky 1989, Feix et al. 2016).

The brief explicitly endorses this: *“A well-justified heuristic pipeline ... is better than a misapplied learned model.”*

2.4 Heuristic grasping strategy

My pipeline segments the fused point cloud using known object positions from MuJoCo, computes PCA-based oriented bounding boxes (OBBs) per object, and selects from five finger presets (power wrap, precision pinch, top grasp, spherical, tripod) based on object shape. I compute palm poses geometrically from the OBB: top-down approaches place the palm above the object center facing down; side approaches place the palm beside the object facing inward.

I rank proposals by grasp-type preference (configured per object in YAML) and observation confidence (point count). I defer reachability filtering via IK to my execution pipeline (Task 4) so I do not duplicate the IK solver.

2.5 IK / planning stack

For Task 4 I use MuJoCo-based damped least-squares IK (`src/planning/ik_solver.py`) with full pose targets (position + orientation) at every waypoint. My execution state machine (`src/planning/executor.py`) runs:

- home -> pre-grasp (horizontal palm, above target)
- pre-grasp -> intermediate (aligned for approach)

- intermediate -> grasp
- finger close (object-specific preset / mug two-stage close)
- lift while maintaining grasp orientation

I check reachability per proposal before execution; I try proposals in score order until I find one that is reachable.

3. Challenges and solutions

3.1 Frame-convention mismatch between RUKA and Piper sources

The RUKA MJCF was authored under a Y-up convention (gravity in $-Y$, ground plane rotated 90° to act as a floor); the Piper MJCF uses MuJoCo’s default Z-up. A simple `<include>` would have produced a model with two contradictory gravity definitions and a tilted floor. I solved it by surgically extracting the hand definition from RUKA and re-embedding it under Piper’s link6 in a freshly-authored Z-up scene file.

3.2 Mount geometry vs. collision approximation mismatch

Initial offsets I derived analytically (5 cm, 6 cm) from the palm collision-box description caused the back-flange of the RUKA palm STL to penetrate link6’s wrist housing — the visual mesh extends further back than the collision box approximation implies. I solved it by empirical visual tuning: `pos.z = 0.10 m` gives a clean fit for me. That was a useful reminder that collision approximations are not always faithful to the visual mesh.

3.3 Single-camera point cloud bias

Early on I used one scene camera, which produced partial point clouds where only front-facing surfaces were visible. My bounding boxes were systematically biased toward the camera — smaller and shifted from the true center. I solved it by adding a second scene camera on the opposite side of the workspace. The fused cloud doubled in size (312k $\rightarrow$ 620k points), OBBs became properly centered, and my grasp proposals improved. Because I wrote the rendering pipeline to take a list of cameras, that change stayed small in code.

3.4 Task-4 execution stability and failure analysis

Two practical issues dominated my execution work:

1. **Reach/orientation coupling at large X reach.** Some lateral orientations that are valid near the robot become infeasible near full extension. I mitigated this with orientation variants (thumb-up vs thumb-down) and scene placement choices that avoid degenerate side-approach geometry.

2. **Contact reliability vs kinematic error.** Stable lifts required targeting the palm site near the knuckle line and tightening approach clearances for side grasps so my fingertips actually contacted the object.

The residual failure I still see is mug grasping (**GRASP_SLIP**), where geometry and orientation constraints make my current top-insert strategy unreliable.

4. Results

On my latest reverification run (May 10, 2026) over all five configured objects:

Object	Result	Lift height	Notes
banana	SUCCESS	+102.6 mm	precision pinch, top approach
cracker_box	SUCCESS	+122.2 mm	power wrap, side approach
mustard_bottle	SUCCESS	+116.1 mm	power wrap, side approach
tennis_ball	SUCCESS	+160.5 mm	spherical grasp, top approach
mug	FAIL	-673.1 mm	GRASP_SLIP

I achieved **4/5 successful grasps (80%)**, which exceeds the requirement of at least 3 successful grasp executions.

5. Demo video (Task 5)

The submission demo is hosted on YouTube: <https://youtu.be/tW-jJHIFZbQ>.

It shows the five-object batch in one continuous quad-view layout (matching the mosaic from `scripts/record_all_grasps.py`): two fixed scene view-points on the table plus two lateral views parented to the palm. Locally I regenerate the underlying MP4 with `visualize_grasps.py --save` followed by `record_all_grasps.py` (artifacts land under gitignored `outputs/`). Per-object narrative and quantitative lift metrics appear in §4 above and in `docs/task4_execution.md`.

6. Key learnings and what I would do differently

Key takeaways for me: - Empirical visual verification beats analytic guessing for geometry-fit decisions where the collision and visual meshes disagree. - A three-level verification harness (symbolic + numerical + physics) caught issues that any single level would have missed; I would reuse that pattern on future sim merges. - For side grasps, small changes in object placement and wrist

orientation can switch a grasp from IK-infeasible to reliable; I now treat layout design as part of the planning problem, not just scene setup.

7. References

- AgileX Piper SDK: https://github.com/agilexrobotics/piper_sdk
- AgileX Piper ROS: https://github.com/agilexrobotics/Piper_ros
- RUKA: <https://github.com/ruka-hand/RUKA>, paper <https://arxiv.org/abs/2504.13165>
- MuJoCo Menagerie: https://github.com/google-deepmind/mujoco_menagerie
- YCB objects: https://github.com/elpis-lab/YCB_Dataset
- Cutkosky, “On Grasp Choice, Grasp Models, and the Design of Hands for Manufacturing Tasks”, IEEE T-RA 1989
- Feix et al., “The GRASP Taxonomy of Human Grasp Types”, IEEE T-HF 2016